UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL

FLUTTER



JULIO 2026-DICIEMBRE 2026

16/10/2026

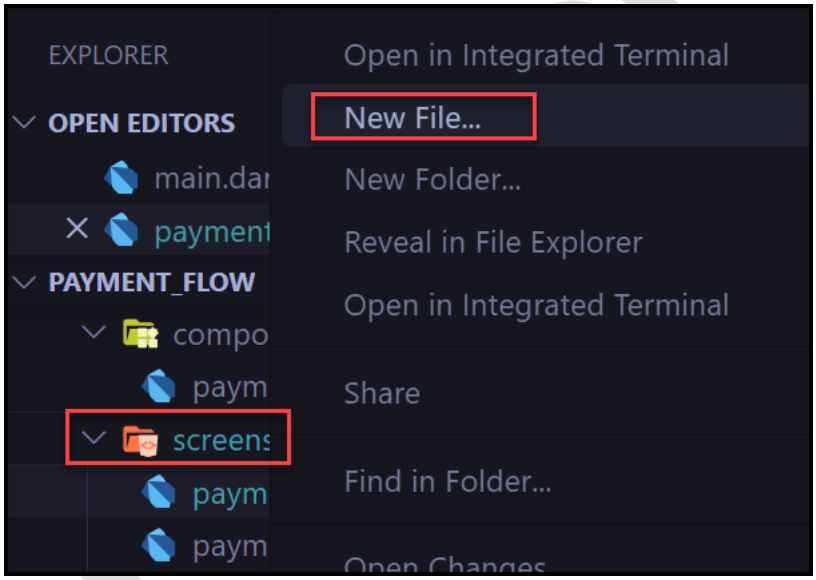
Ejercicio # 1

En esta práctica vamos a empezar con el diseño de la segunda pantalla que tiene relación con los detalles de pago para la aplicación en Flutter.

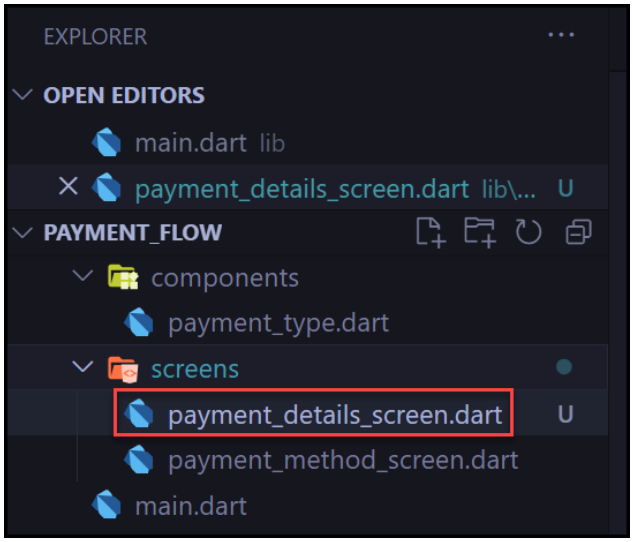
Detalles de pago #1

1. En las siguientes prácticas se va a diseñar la segunda pantalla que tiene relación con los detalles del pago. Para empezar vaya a la carpeta **screens** del proyecto, cree un **nuevo archivo** con el nombre **payment_details_screen.dart** para diseñar la pantalla de detalles del pedido para la aplicación.

Cree un nuevo archivo en la carpeta screens.



Archivo creado.



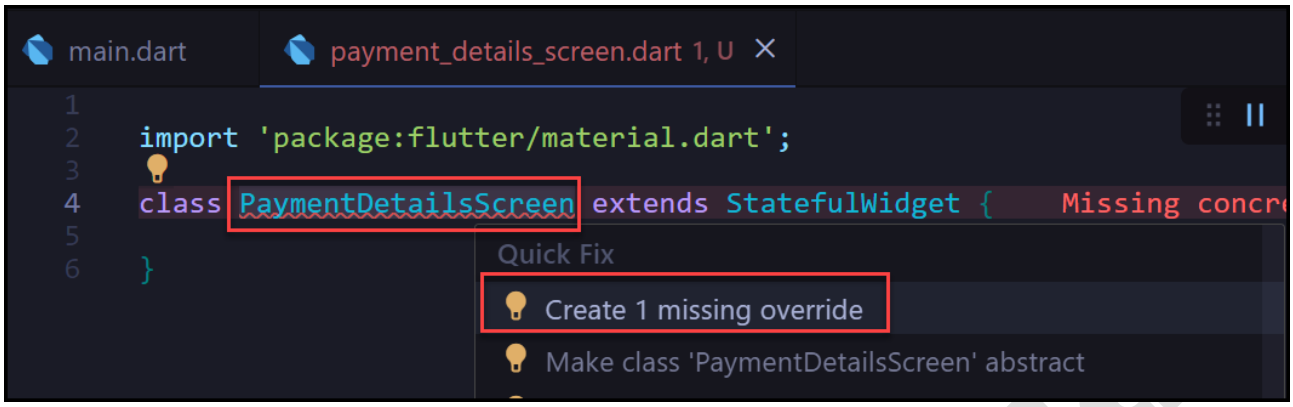
2. Dentro del archivo anterior **cree** la clase **PaymentDetailsScreen** heredada de la clase **StatefulWidget**, luego importe el paquete de Material Design (**material)**, donde están todos los widgets para crear la **UI** para Android específicamente.



Nota	• Lo heredamos de la clase **StatefulWidget** porque alguno de los **componentes** que va a tener la clase van a cambiar la **UI,** por lo tanto, es necesario administrar el estado (los cambios).

3. **Implemente** el método **createState** en la clase creada, para esto ponga el cursor en la clase **PaymentDetailsScreen**, **presione** las teclas **CONTROL + PUNTO** (Control + .), de la ventana emergente **seleccione** la opción **create missing override**.

Seleccione la opción indicada.

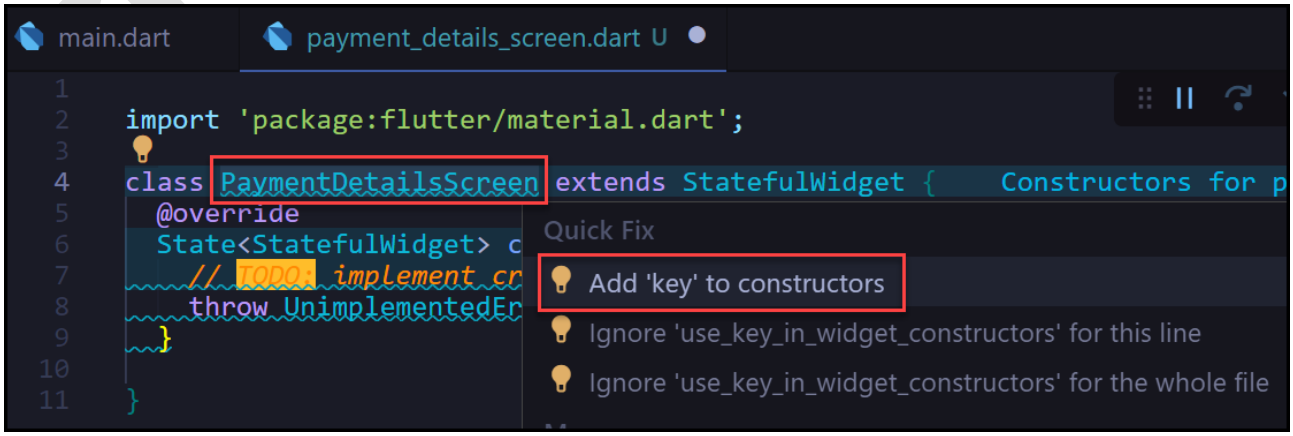


Se crea el método **createState**.



4. **Ponga** nuevamente el **cursor** sobre la clase **PaymentDetailsScreen**, **presione** las teclas **CONTROL + PUNTO** (Control + .) y de la ventana emergente **seleccione** la opción **Add 'key' to constructors**, para crear el constructor de la clase y agregar a este método el parámetro **key**.

Seleccionamos la opción indicada.



Se crea el **constructor** con el parámetro **key**.

```dart
import 'package:flutter/material.dart';
class PaymentDetailsScreen extends StatefulWidget {
  const PaymentDetailsScreen({super.key});
  @override
  State<StatefulWidget> createState() {
    // TODO: implement createState
    throw UnimplementedError();
  }
}
```

5. En el mismo archivo, al final **cree** la clase privada **_PaymentDetailsScreenState** que debe **extender** de la clase **State** la cual se va a utilizar para **administrar el estado** de la clase **PaymentDetailsScreen** creada en el **punto 2**. Esto se muestra en la siguiente figura.

Cree la clase **_PaymentDetailsScreenState**:

```dart
import 'package:flutter/material.dart';

class PaymentDetailsScreen extends StatefulWidget {
  const PaymentDetailsScreen({super.key});

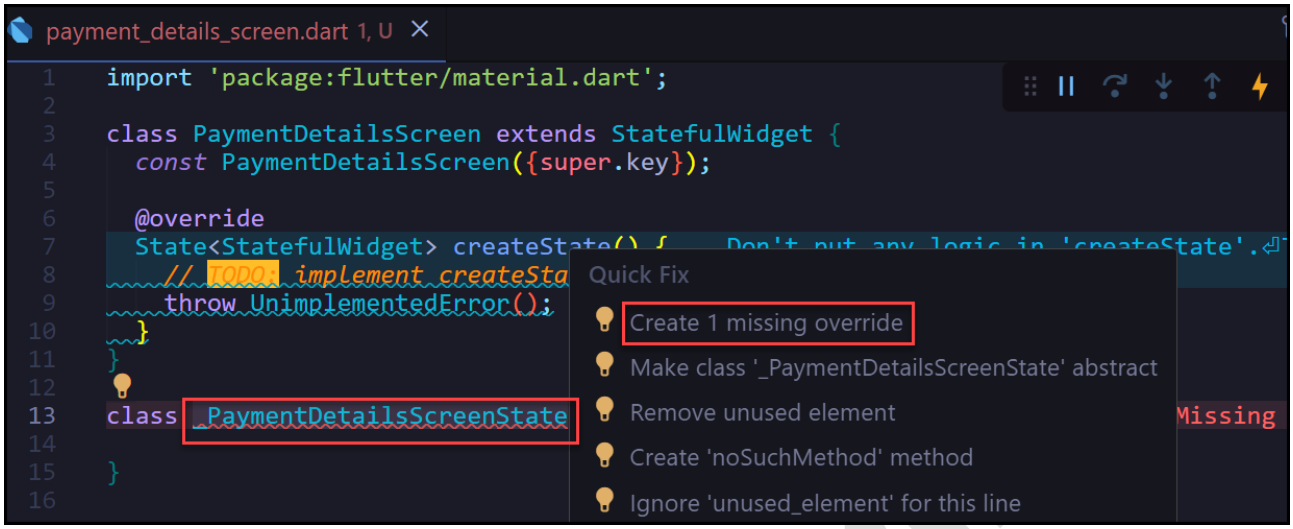
  @override
  State<StatefulWidget> createState() {
    // TODO: implement createState
    throw UnimplementedError();
  }
}

class _PaymentDetailsScreenState extends State<PaymentDetailsScreen> {

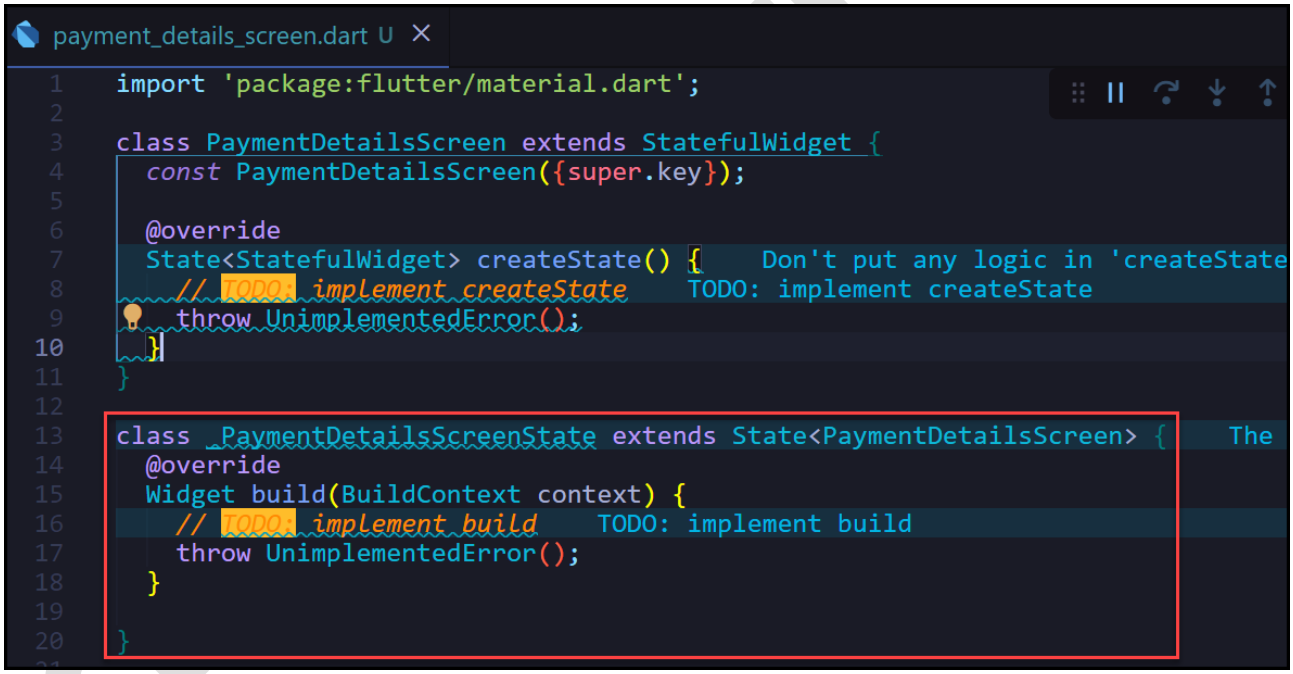
}
```

6. Ahora **implemente** el método **build** para la clase creada, para esto ponga el cursor en la clase privada **_PaymentDetailsScreenState**, **presione** las teclas **CONTROL + PUNTO** (Control + .), de la ventana emergente **seleccione** la opción **Create 1 missing override** para generar el método **build**.

Ponga el cursor en la clase privada **_PaymentDetailsScreenState** y seleccione la opción indicada.



Se genera el método **build**.



7. En el método **createState** de la primera clase (**PaymentDetailsScreen**) **borre** el código y **retorne** una instancia de la clase de tipo **_PaymentDetailsScreenState**, esta clase es la que va a **administrar el estado** de la pantalla. En el método **build** de la segunda clase (**_PaymentDetailsScreenState**) **borre** el código y **retorne** un widget de tipo **Scaffold** el cual se utiliza para construir el esquema principal de la **UI** de la pantalla, esto se muestra a continuación.

payment_details_screen.dart U

```dart
import 'package:flutter/material.dart';

class PaymentDetailsScreen extends StatefulWidget {
  const PaymentDetailsScreen({super.key});

  @override
  State<StatefulWidget> createState() {

    return _PaymentDetailsScreenState();

  }
}

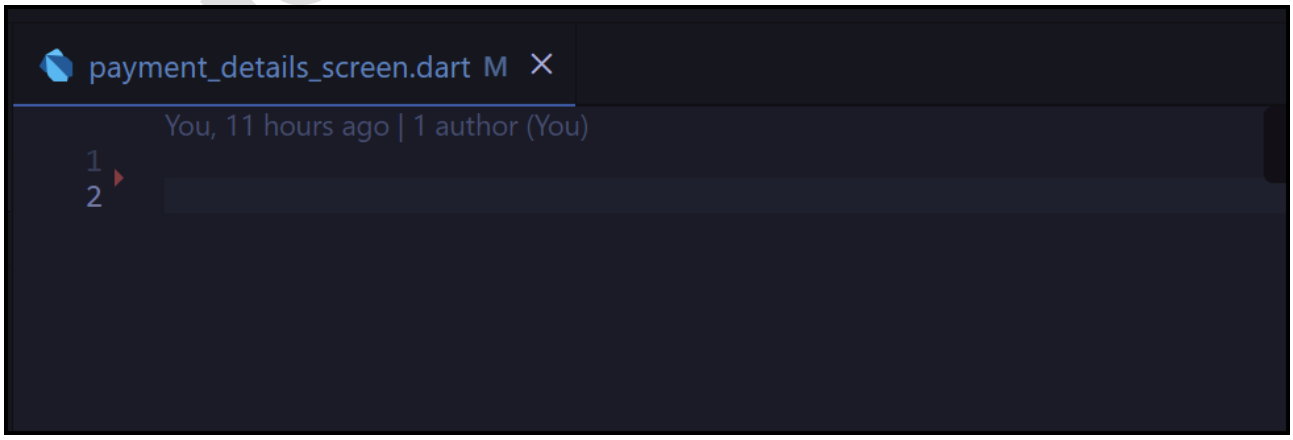
class _PaymentDetailsScreenState extends State<PaymentDetailsScreen> {
  @override
  Widget build(BuildContext context) {

    return const Scaffold();

  }
}
```

8. Una forma **alternativa** de crear un widget de tipo **StatelessWidget** o de tipo **StatefulWidget**, es utilizar los **snippets** de código que se añaden al instalar las diferentes extensiones (**plugins**) para Flutter en Visual Studio Code, por ejemplo, con la extensión **Awesome Flutter Snippets**.

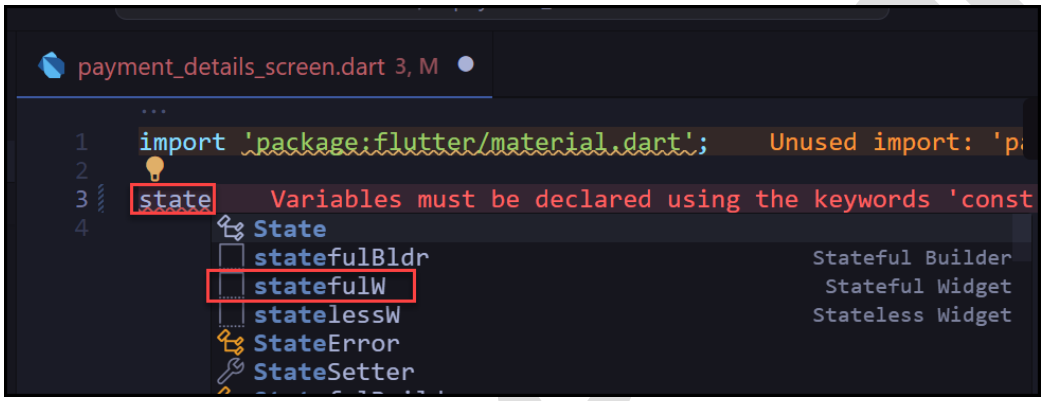
9. Por ejemplo, vamos a repetir la creación del widget **PaymentDetailsScreen** utilizando el snippets **Awesome Flutter Snippets**. Por lo tanto, **borre todo el contenido** del archivo **payment_details_screen**. Esto se muestra en la siguiente pantalla.



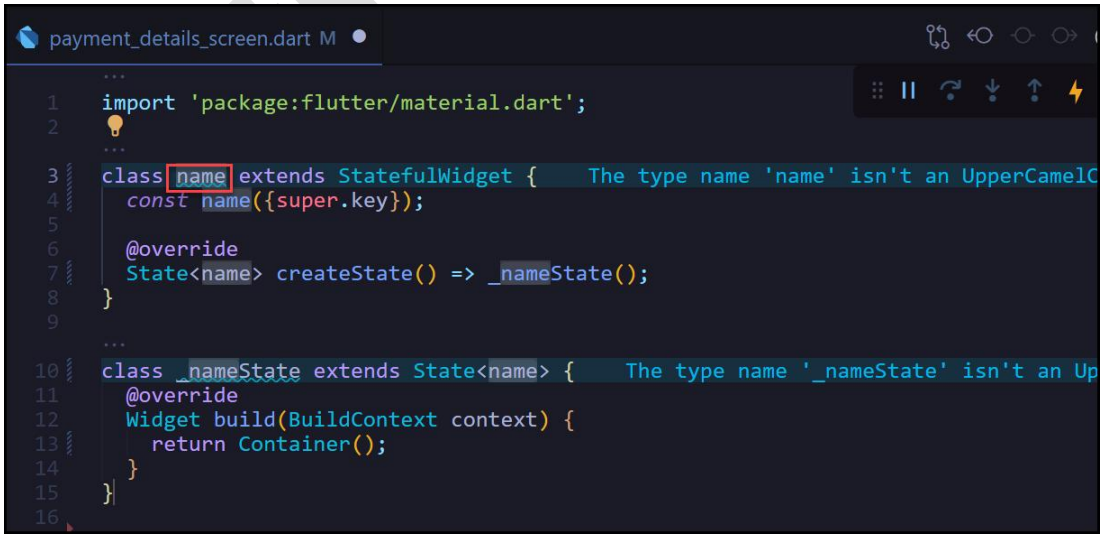
10. Importe el paquete "**material.dart**" para utilizar Material Design.



11. **Escriba** el texto **statefulW** que es un **shortcut** agregado al instalar el complemento (plugin). Al **escribir** el texto indicado se muestra la siguiente ventana.



12. De la lista emergente anterior **seleccione** el item **statefulW** para crear el formato para un widget de tipo **StatefulWidget**. Esto crea el siguiente código.



Nota	• Con el shortcut **StatelessW** se crea el formato para un widget de tipo **StatelessWidget.**

13. Donde se muestra el texto **name** (que se muestra parpadeando) con cuidado **digite** el texto **PaymentDetailsScreen**, esto **reemplaza** el texto **name** con el texto ingresado en **todas líneas** donde aparezca el texto **name**. El resultado es el siguiente.



14. Realice los cambios indicados en la siguiente figura. En la primera clase **PaymentDetailsScreen** en el método **createState** cambie la expresión **lamba** por un **return**. En la **segunda** clase de tipo privada **_PaymentDetailsScreenState** cambie el **return** para que regrese un **Scaffold**, el resultado se muestra a continuación.



Nota	• El resultado es el mismo obtenido en el **punto 7** de esta práctica. Por lo tanto, depende del desarrollador la forma de crear el código para los widgets de tipo **StatelessWidget** o **StatefulWidget**, sea de forma manual o con la ayuda de complementos (**plugins**) que genaran snippets (fragmentos de código).

Vamos a continuar con la práctica.

15. Como se creó una nueva pantalla, hay que **establecer una nueva ruta** para que se puede navegar hacia esta **pantalla**. Para eso vaya al archivo **main.dart**, dentro del método **build**, en la propiedad **routes**, digite el siguiente código para agregar la ruta de navegación para la pantalla.

```
main.dart M
18          appBarTheme: const AppBarTheme(        You, 12 hours a
19            color: Colors.white,
20            elevation: 0,
21            iconTheme: IconThemeData(color: Color(0xff050060)),
22          ), // AppBarTheme
23          scaffoldBackgroundColor: Colors.white,
24
25          elevatedButtonTheme: ElevatedButtonThemeData(
26            style: ElevatedButton.styleFrom(
27                backgroundColor: const Color(0XFF34495E)),
28          ), // ElevatedButtonThemeData
29
30        ), // ThemeData
31        initialRoute: '/',
32        title: 'Flujo de pago',
33        routes: {
34          '/': (context) => const PaymentMethodScreen(),
35
36          '/payment_details': (context) => const PaymentDetailsScreen(),
37
38        },
39      ); // MaterialApp
40    }
41  }
42
```

Donde:

- **payment_details**: Aquí se puede digitar cualquiera texto, aunque es mejor que sea un texto que haga referencia al nombre de la pantalla que se desea mostrar.
- **PaymentDetailsScreen**: Es una instancia de la pantalla que se desea mostrar.

16. El código anterior debe agregar al **inicio** del archivo **main.dart** la importación del **archivo** donde está definido la clase **PaymentDetailsScreen**. El archivo **main.dart** debería tener las siguientes referencias.



Nota	• Si **no se importó** la referencia automáticamente entonces hay que **importar** el archivo **manualmente.**

17. La pantalla se debe mostrar al **presionar** el botón **Siguiente** de la primera pantalla. Por lo tanto abra el archivo **payment_method_screen.dart**, luego vaya a la clase **PaymentMethodState**, en el método **build**, en la propiedad **floatingActionButton**, dentro del **ElevatedButton**, en la propiedad **onPressed** de este **botón**, digite el siguiente código para llamar al método llamado **showDetails** con el parámetro **context**, este parámetro permite poder referenciar a cualquier widget del árbol de widgets creados en la pantalla.



18. Al final de la **clase** cree el método **showDetails**.

payment_method_screen.dart M

```
103            child: const Text(
104              'SIGUIENTE',
105            ), // Text
106          ), // ElevatedButton       You, 9 hours ago • Terminada practica 006
107          floatingActionButtonLocation: FloatingActionButtonLocation.centerFloat,
108        ); // Scaffold
109      }
110
111      void _onPaymentChange(PaymentMethod value) {
112        setState(() {});
113        paymentSelected = value;
114      }
115
116      void showDetails(BuildContext context) {
117        Navigator.pushNamed(context, '/payment_details');
118      }
119    }
120
```

- El parámetro de tipo **BuildContext** permite hacer referencia a cualquier widget del árbol de widgets del proyecto.
- En Flutter, la navegación de una pantalla a otra es posible gracias a clase **Navigator**, este es un widget simple que **mantiene una pila de Rutas**, o en términos más simples, un historial de **pantallas / páginas visitadas**.
- Con la ayuda del método **pushNamed**, se puede navegar hacia cualquier pantalla cuya ruta está definida en el archivo **main.dart** .

Nota	• El texto del **segundo parámetro** (‘**/payment_details**’) debe ser **exactamente el mismo** que digitado en la propiedad **routes** del método **build,** clase **MyApp**, del archivo **main.dart,** caso contrario el código **genera un error.**

19. El método **showDetails** es de tipo **privado** porque la clase **_PaymentMethodState** utiliza al inicio el **guion bajo** para definir el nombre, es decir es **privada**, y por lo tanto el método **showDetails** también es privado por defecto. Esto significa que en el

método **_onPaymentChange** no hace falta el **guion bajo** para que sea de alcance **privado**.



20. Por lo tanto, **borre** el **guion bajo** del **nombre** este método y **cambie todas las líneas** donde se llame a este **método** para tener el código de acuerdo las **convenciones de nomenclatura** recomendadas por **Flutter**. Esto se muestra a continuación.

Flutter

payment_method_screen.dart M

```dart
                'Seleccione uno de los métodos de pago',
                style: TextStyle(
                  fontSize: 12,
                  color: Color(0xff6C738A),
                ), // TextStyle
              ), // Text
              const SizedBox(
                height: 20,
              ), // SizedBox
              PaymentMethodWidget(
                icon: const Icon(
                  FontAwesomeIcons.creditCard,
                  color: Color(0xffCCB375),
                ), // Icon
                title: 'Tarjeta de crédito',
                subTitle: 'Pague con Mastercard o Visa',
                value: PaymentMethod.creditCard,
                selectedValue: paymentSelected,
      You, 28 seconds ago • Uncommitted changes
                paymentCallBack: onPaymentChange,

              ), // PaymentMethodWidget
              PaymentMethodWidget(
                icon: const Icon(
                  FontAwesomeIcons.buildingColumns,
                  color: Color(0xff00A7D9),
                ), // Icon
                title: 'Transferencia bancaria',
                subTitle: 'Pague directo desde su cuenta',
                value: PaymentMethod.bank,
                selectedValue: paymentSelected,

                paymentCallBack: onPaymentChange,

              ), // PaymentMethodWidget
              PaymentMethodWidget(
                icon: const Icon(
                  FontAwesomeIcons.paypal,
                  color: Color(0xff0D47A1),
                ), // Icon
                title: 'Paypal',
                subTitle: 'La forma más rápida y segura de enviar dinero',
                value: PaymentMethod.paypal,
                selectedValue: paymentSelected,

                paymentCallBack: onPaymentChange,

              ), // PaymentMethodWidget
              PaymentMethodWidget(
                icon: const Icon(
                  FontAwesomeIcons.bitcoin,
                  color: Color(0xff000000),
                ), // Icon
                title: 'Billetera de bitcoin',
                subTitle: 'Enviar la cantidad a nuestra billetera Bitcoin',
                value: PaymentMethod.bitcoin,
                selectedValue: paymentSelected,

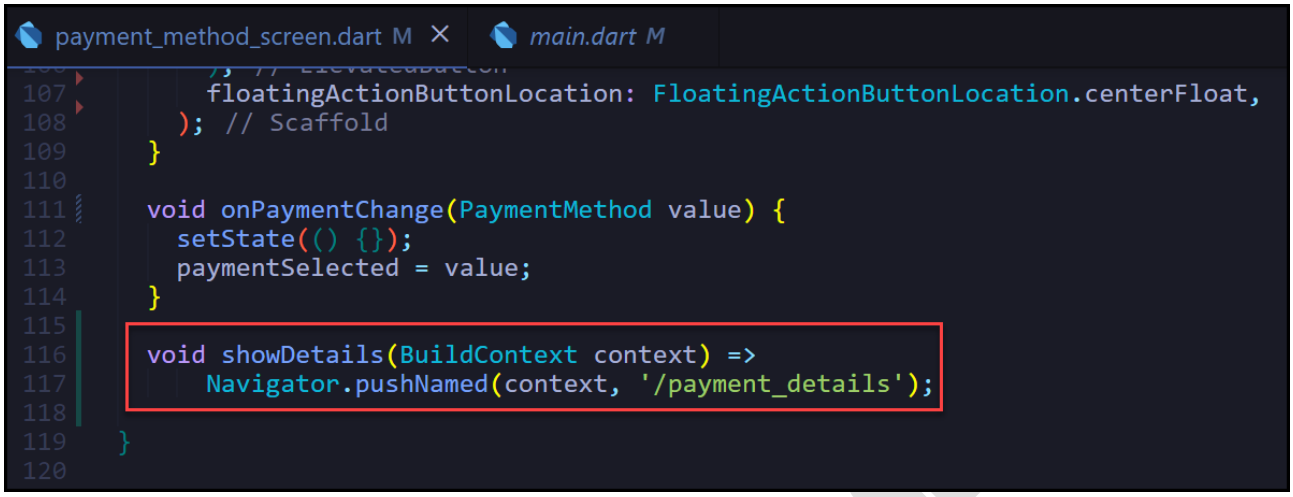
                paymentCallBack: onPaymentChange,
              ), // PaymentMethodWidget

            ], // <Widget>[]
          ), // Column
        ), // Padding
        floatingActionButton: ElevatedButton(
          onPressed: () {
            showDetails(context);
          },
          style: ElevatedButton.styleFrom(
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(25),
            ), // RoundedRectangleBorder
          ),
          child: const Text(
            'SIGUIENTE',
          ), // Text
        ), // ElevatedButton
        floatingActionButtonLocation: FloatingActionButtonLocation.centerFloat
      ); // Scaffold
    }

    void onPaymentChange(PaymentMethod value) {
      setState(() {});
      paymentSelected = value;
    }

    void showDetails(BuildContext context) {
      Navigator.pushNamed(context, '/payment_details');
    }
  }
```

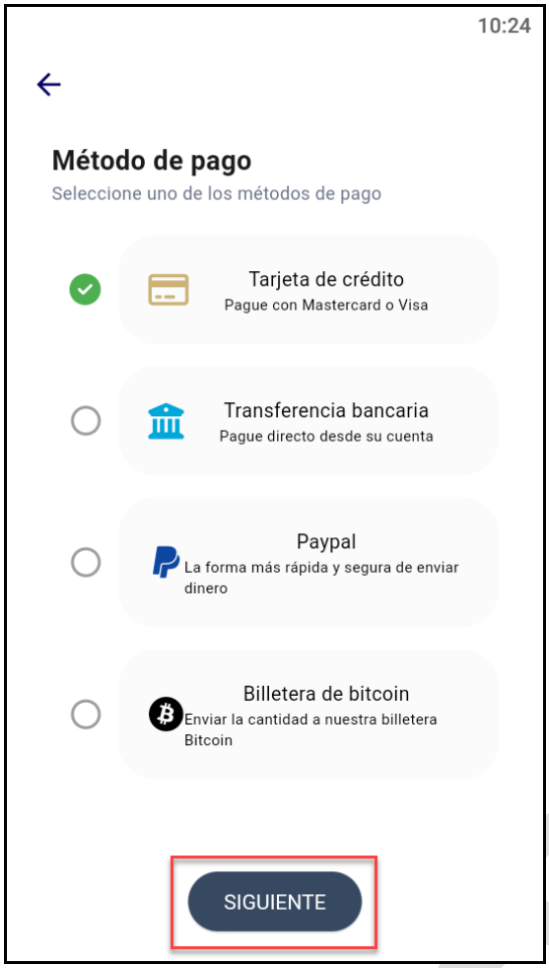
21. Puesto que el método **showDetails** en su cuerpo únicamente tiene **una línea de código**, este se puede escribir como una expresión **lamba**, de la siguiente forma.



22. También la propiedad **onPressed** del **floatingActionButton** donde se llama al método **showDetails**, se compone de una sola línea, por lo tanto, se puede escribir como una expresión **lamba**, de la siguiente forma.

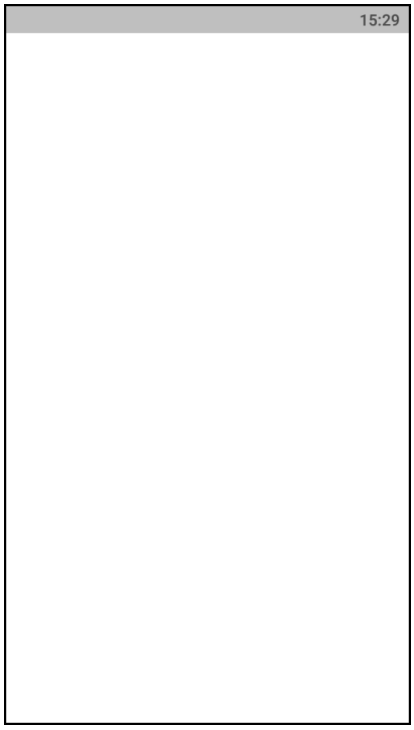


23. **Ejecute** y **pruebe** el proyecto.



24. **Presione** el botón **Siguiente**. Al **presionar** en el botón se muestra la pantalla **PaymentDetailsScreen** especificada en el código del **botón**.

FlutterClay Aldás



25. La pantalla por el momento está **vacía**, en las próximas prácticas se va a crear la **UI** para esta pantalla.

Lecturas útiles para revisar y profundizar en los temas abordados en esta práctica.

BuildContext class

https://api.flutter.dev/flutter/widgets/BuildContext-class.html

Navigator class

https://api.flutter.dev/flutter/widgets/Navigator-class.html

flutterClay Aldás